Algoritmos de optimización - Trabajo Práctico

Nombre y Apellidos: Karen Nattaly Valero González
Url: https://colab.research.google.com/drive/1fuYFuiUOWWI0c4RtXplVd3338wA8q8kM?usp=sharing
Google Colab: https://github.com/knvalerog22/Algoritmos_optimizacion/tree/main/Trabajo

Problema: Sesiones de doblaje

Descripción del problema:

Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas

Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben.

- **Restricción**: No es posible grabar más de 6 tomas por día
- **Objetivo**: El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible.
- **Datos**:
    1. Número de actores: 10
    2. Número de tomas: 30
    3. Actores / Tomas: 1 indica que el actor participa en la toma , 0 en caso contrario

Matriz de Participación — Actores por Toma

Toma	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	Total
1	1	1	1	1	1	0	0	0	0	0	**5**
2	0	0	1	1	1	0	0	0	0	0	**3**
3	0	1	0	0	1	0	1	0	0	0	**3**
4	1	1	0	0	0	0	1	1	0	0	**4**
5	0	1	0	1	0	0	0	1	0	0	**3**
6	1	1	0	1	1	0	0	0	0	0	**4**
7	1	1	0	1	1	0	0	0	0	0	**4**
8	1	1	0	0	0	1	0	0	0	0	**3**
9	1	1	0	1	0	0	0	0	0	0	**3**
10	1	1	0	0	0	1	0	0	1	0	**4**
11	1	1	1	0	1	0	0	1	0	0	**5**
12	1	1	1	1	0	1	0	0	0	0	**5**
13	1	0	0	1	1	0	0	0	0	0	**3**
14	1	0	1	0	0	1	0	0	0	0	**3**
15	1	1	0	0	0	0	1	0	0	0	**3**
16	0	0	0	1	0	0	0	0	0	1	**2**
17	1	0	1	0	0	0	0	0	0	0	**2**
18	0	0	1	0	0	1	0	0	0	0	**2**
19	1	0	1	0	0	0	0	0	0	0	**2**
20	1	0	1	1	1	0	0	0	0	0	**4**
21	0	0	0	0	0	1	0	1	0	0	**2**
22	1	1	1	1	0	0	0	0	0	0	**4**
23	1	0	1	0	0	0	0	0	0	0	**2**
24	0	0	1	0	0	1	0	0	0	0	**2**
25	1	1	0	1	0	0	0	0	0	1	**4**
26	1	0	1	0	1	0	0	0	1	0	**4**
27	0	0	0	1	1	0	0	0	0	0	**2**
28	1	0	0	1	0	0	0	0	0	0	**2**
29	1	0	0	0	1	1	0	0	0	0	**3**
30	1	0	0	1	0	0	0	0	0	0	**2**
TOTAL	**22**	**14**	**13**	**15**	**11**	**8**	**3**	**4**	**2**	**2**	**94**

Modelo

¿Como represento el espacio de soluciones?

El número mínimo de días de grabación está determinado por la restricción del problema:

30 tomas / 6 tomas por día = 5 días mínimo.

Si aplicamos combinatoria para saber cuántas formas distintas existen de repartir las 30 tomas en esos 5 días, tenemos:

```
Elegir 6 tomas para el Día 1: C(30,6) =    593.775
Elegir 6 tomas para el Día 2: C(24,6) =    134.596
```

```
Elegir 6 tomas para el Día 3: C(18,6) =     18.564
Elegir 6 tomas para el Día 4: C(12,6) =        924
Elegir 6 tomas para el Día 5: C(6,6)  =          1

Total solo para 5 días exactos:593.775 × 134.596 × 18.564 × 924 × 1 ≈ 1.37 × 10^18 combinaciones.
```

Este cálculo representa solo el caso de 5 días exactos. Si se tienen en cuenta otra soluciones con más días, el espacio a considerar será mayor.

El espacio de soluciones de este problema supera las 10^18 combinaciones posibles. Asumiendo una evaluación de 10^9 soluciones por segundo (estimación estandar basada en procesadores modernos de ~3GHz), el tiempo necesario para explorar todo el espacio seria ≈ 31 años ≈ 10^18 / 10^9 ≈ 10^9 segundos.

Por lo cual usar un algoritmo como fuerza bruta se hace inviable, y se justifica el uso de una metaheurística.

¿Cual es la función objetivo?

La función objetivo mide el costo total de convocar actores a lo largo de todos los días de grabación. Dado que cada actor cobra una tarifa fija por día que asiste al estudio, independientemente del número de toma que grabe ese día, el costo de un día equivale al número de actores distintos convocados ese día.

Toma	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	Total
1	1	1	1	1	1	0	0	0	0	0	**5**
2	0	0	1	1	1	0	0	0	0	0	**3**
3	0	1	0	0	1	0	1	0	0	0	**3**

```
- Actores T1 = {1,2,3,4,5}
- Actores T2 = {3,4,5}
- Actores T3 = {2,5,7}

Unión    = {1,2,3,4,5,7}
Costo    = 6  ->  6 actores para 3 tomas

S = Solución (vector de asignación)
d = Cada día de grabación
D = Conjunto de días utilizados en la solución
T_d = Conjunto de tomas asignadas al día d
A(t) = Conjunto de actores que participan en la toma t
U = Unión de conjuntos (sin repetir)
| | = Número de elementos


- El costo de un día d es:

    Costo(d) = |U A(t)| Para todo t ∈ T_d  (Representa la unión de todas las tomas del día)

- La función objetivo a minimizar es:

   F(S) = ∑ Costo(d)  Para todo d ∈ D
        = ∑  |u A(t)|
```

¿Cómo implemento las restricciones?

Restricción : No es posible grabar más de 6 tomas por día

```
Para todo día d: |T_d| ≤ 6

T_d = Conjunto de tomas asignadas al día d
```

Este problema representa una restricción dura, por lo cual ninguna solución valida puede violarla bajo ninguna circunstancia.

Para garantizar que el Algoritmo genético trabaje siempre con soluciones validas, se aplican dos estrategias complementarias:

1. Construcción válida: Generando soluciones iniciales que ya cumplan la restricción desde el principio. Por lo cual al ir asignando tomas, se verificará que el día destino no supere las 6 tomas.
   - Si día d tiene menos de 6 tomas -> Se asigna
   - Si día d tiene 6 tomas -> Busca otro día
   - Si todos los días están llenos -> Crea un día nuevo
2. Reparación: Permite que la mutación genere soluciones inválidas, para repararlas inmediatamente despues.

   Despues del cruce: Día 3 tiene 8 tomas (viola la restricción)
   - Toma 2 tomas del día 3
   - Mover a otro día con espacio disponible
   - Solución reparada

Utilizando estas dos estrategías se garantiza que el algortimo nunca evalúe soluciones inválidas.

Análisis

¿Qué complejidad tiene el problema?.

Para ver como crece el tiempo de ejecución en función del tamaño de nuestro proyecto, tenemos en consideración lo siguiente:

En nuestro problema, la operación principal es evaluar el costo de una solución, lo que implica recorrer todas las tomas y sus actores.

```
n = Número de tomas (30)
m = Número de actores (10)
G = Número de generaciones(cuántas veces evoluciona la poblaciión, para este probema será de 500)
P = Tamaño de la población (cuántas soluciones se evalúan por generación, para este problema será de 100)

 O(n x m) = 300 operaciones por solución

Debido a que en cada generación (G), se evalúan P soluciones, y cada una cuesta O(n x m), tenemos:

 O(G x P x n x m) = 500 x 100 x 30 x 10 = 1.5 x 10 ^ 7 operaciones
```

Diseño

¿Qué técnica se utiliza? ¿Por qué?

Debido a que el espacio de soluciones supera las 10^18 combinaciones posibles, se descarta cualquier método exacto o de fuerza bruta y se opta por una metaheurística.

Se seleccionó el Algoritmo Genético por las siguientes razones:

1. Respaldo en literatura académica: Se ha observado que es uno de los algortimos más utilizados para este tipo de problema (Nordström & Tufekci, 1994) .
2. Representacion Natural: La solución se codifica directamente como un vector de 30 enteros, donde cada posición representa una toma y su valor para el día asignado. Esta estructura reprsenta de forma inmediata un cromosoma del Algoritmo Genético, sin necesidad de transformaciones adicionales.

   Ejemplo:

   S = [3,2,1,3....] S Cromosoma, en donde cada posición una toma diferente y cada valor representa el día de grabación de esa toma.

   "La toma 1, va al día 3"
3. Al mantener una población de soluciones (100), el AG explora múltiples regiones del espacio de búsqueda, reduciendo el riesgo de quedar atrapado en óptimos locales.
4. Los tres operadores (selección, cruce y mutación), trabajan en conjunto para equilibrar la búsqueda.

```
# Librerias
import numpy as np
import random
import matplotlib.pyplot as plt
```

Datos de entrada del problema

```
'''
DATOS DE ENTRADA
  N_TOMAS   = 30   # número de tomas
  N_ACTORES = 10   # número de actores
  MAX_TOMAS_DIA = 6
'''

# Matriz de participación [30 tomas x 10 actores]
# M[i][j] = 1 si el actor j participa en la toma i
M = np.array([
    [1,1,1,1,1,0,0,0,0,0],  # Toma 1
    [0,0,0,1,1,1,0,0,0,0],  # Toma 2
    [0,1,0,1,0,1,0,1,0,0],  # Toma 3
    [1,1,0,0,1,1,0,1,0,0],  # Toma 4
    [0,1,0,0,0,0,0,1,0,0],  # Toma 5
    [1,1,0,1,0,0,0,0,0,0],  # Toma 6
    [1,0,0,0,1,1,0,0,0,0],  # Toma 7
    [0,1,0,1,1,0,0,0,0,0],  # Toma 8
    [0,0,0,0,1,0,1,1,0,0],  # Toma 9
    [1,0,0,1,0,0,0,1,0,0],  # Toma 10
    [1,1,0,0,0,0,0,0,0,1],  # Toma 11
    [0,0,0,1,1,0,1,0,0,0],  # Toma 12
    [0,0,0,1,0,0,1,1,0,0],  # Toma 13
    [1,0,0,0,0,1,0,1,0,0],  # Toma 14
    [0,0,0,0,1,0,0,0,1,1],  # Toma 15
    [1,1,0,0,0,0,0,0,0,0],  # Toma 16
    [0,0,0,1,1,0,0,0,0,1],  # Toma 17
    [0,0,1,0,0,1,0,0,1,0],  # Toma 18
    [0,0,0,0,1,0,1,0,0,0],  # Toma 19
```

```
    [1,0,0,0,0,0,0,1,1,0],  # Toma 20
    [0,0,0,0,0,1,0,1,1,0],  # Toma 21
    [1,0,0,1,0,0,0,0,1,0],  # Toma 22
    [0,0,1,0,0,0,1,0,1,0],  # Toma 23
    [0,0,1,0,1,0,0,0,0,1],  # Toma 24
    [1,0,0,1,0,0,0,0,0,1],  # Toma 25
    [0,1,0,0,0,1,0,0,1,0],  # Toma 26
    [0,0,1,0,0,0,0,1,0,1],  # Toma 27
    [0,0,0,1,0,1,0,0,0,1],  # Toma 28
    [1,0,0,0,0,0,1,0,0,1],  # Toma 29
    [0,0,1,0,0,0,1,0,1,0],  # Toma 30
])

#Se definen las constantes del problema
N_TOMAS   = 30   # número de tomas
N_ACTORES = 10   # número de actores
MAX_TOMAS_DIA = 6
```

Función de costo (Fitness)

Implementa la funcion objetivo:

```
F(S) = Σ | ∪ A(t) | para todo día d

- Recorre cada toma y la agrupa por día
- Para cada día calcula los actores únicos convocados
- Retorna la suma total (la cual queremos minimizar)
```

```
def calcular_costo(solucion):
    """
    Recibe un vector de 30 enteros donde solucion[i] = día asignado a la toma i.
    Retorna el costo total = suma de actores únicos convocados por día.
    """
    dias = {} # Diccionario: día -> conjunto de actores
    for toma, dia in enumerate(solucion):
        if dia not in dias:
            dias[dia] = set()
        for actor in range(N_ACTORES):
            if M[toma][actor] == 1:
                dias[dia].add(actor)

    costo_total = sum(len(actores) for actores in dias.values())
    return costo_total

def es_valida(solucion):
    """
    Verifica que ningún día tenga más de MAX_TOMAS_DIA tomas (6)
    """
    conteo = {}
    for dia in solucion:
        conteo[dia] = conteo.get(dia, 0) + 1
    return all(v <= MAX_TOMAS_DIA for v in conteo.values())
```

```
# prueba de función calcular_costo
solucion_test = [1,2,1,3,2,1,4,3,2,5,
                 1,2,3,4,5,1,2,3,4,5,
                 1,2,3,4,5,1,2,3,4,5]

print(f"Costo solución test: {calcular_costo(solucion_test)}")

valida = es_valida(solucion_test)
print(f"¿Solución válida?:   {valida}")

# ¿Cuántas tomas tiene cada día'
conteo = {}
for dia in solucion_test:
    conteo[dia] = conteo.get(dia, 0) + 1
print(f"Tomas por día:       {dict(sorted(conteo.items()))}")
print(f"Total tomas: {sum(conteo.values())}")
```

```
Costo solución test: 44
¿Solución válida?:   False
Tomas por día:       {1: 7, 2: 7, 3: 6, 4: 5, 5: 5}
Total tomas: 30
```

Generación de población inicial

Para que el GA tenga de dónde partir, se genera una población inicial de P soluciones aleatorias válidas

```
Cada solución se construye asignando las 30 tomas a días
de forma aleatoria, respetando desde el principio la
restricción de máximo 6 tomas por día:

- Si el día tiene menos de 6 tomas → se asigna
- Si el día tiene 6 tomas → busca otro día
- Si todos los días están llenos → crea un día nuevo
```

```
Esto garantiza que todas las soluciones iniciales sean válidas antes de entrar al GA.
```

```python
def generar_solucion():
    """
    Genera una asignación aleatoria válida de tomas a días.
    Se garantiza que ningún día supere MAX_TOMAS_DIA tomas.
    Retorna: Vector de 30 enteros

    """
    solucion = [0] * N_TOMAS
    capacidad = {}  # día -> tomas asignadas

    tomas_shuffled = list(range(N_TOMAS))
    random.shuffle(tomas_shuffled)

    dia_actual = 1
    for toma in tomas_shuffled:
        # Buscar un día con espacio disponible
        asignado = False
        dias_disponibles = [d for d, c in capacidad.items() if c < MAX_TOMAS_DIA]
        random.shuffle(dias_disponibles)

        for dia in dias_disponibles:
            solucion[toma] = dia
            capacidad[dia] = capacidad.get(dia, 0) + 1
            asignado = True
            break

        if not asignado:
            # Crear nuevo día
            solucion[toma] = dia_actual
            capacidad[dia_actual] = 1
            dia_actual += 1

    return solucion

def generar_poblacion(tam_poblacion):
    return [generar_solucion() for _ in range(tam_poblacion)]

```

```python
#Test

sol = generar_solucion()

conteo = {}
for dia in sol:
    conteo[dia] = conteo.get(dia, 0) + 1

print(f"Solución generada:  {sol}")
print(f"Tomas por día:      {dict(sorted(conteo.items()))}")
print(f"Total tomas:        {sum(conteo.values())}")
print(f"¿Válida?:           {es_valida(sol)}")
print(f"Costo:              {calcular_costo(sol)}")

poblacion = generar_poblacion(100)
invalidas = sum(1 for s in poblacion if not es_valida(s))
print(f"\nPoblación de 100 → Inválidas: {invalidas}")
print(f"Costo promedio: {sum(calcular_costo(s) for s in poblacion)/len(poblacion):.1f}")
```

```
Solución generada:  [3, 4, 2, 4, 5, 2, 5, 2, 5, 5, 1, 5, 3, 4, 4, 1, 1, 1, 3, 3, 1, 5, 2, 3, 4, 2, 3, 1, 2, 4]
Tomas por día:      {1: 6, 2: 6, 3: 6, 4: 6, 5: 6}
Total tomas:        30
¿Válida?:           True
Costo:              46

Población de 100 → Inválidas: 0
Costo promedio: 45.2
```

La solución del test anterior es un punto de partida para el GA.

El costo de 48 indica que en total se convocan 48 actores-día sumando todos los días. El GA buscará minimizar este valor.

Selección por torneo

La selección determina qué individuos de la población tienen derecho a reproducirse para generar la siguiente generación.

```
Se utiliza selección por torneo:
1. Se eligen k individuos al azar de la población
2. Se comparan sus costos
3. Gana el de menor costo → se convierte en padre

Lo cual garantiza que los mejores individuos tengan más probabilidad de reproducirse, sin descartar completamente a
```

```python
def seleccion_torneo(poblacion, k=3):
    """
    Selecciona k individuos al azar y retorna el mejor (menor costo).
    """
```

```
    competidores = random.sample(poblacion, k)
    return min(competidores, key=calcular_costo)
```

```
#Test
poblacion = generar_poblacion(10)

# Mostrar costos de toda la población
costos = [calcular_costo(ind) for ind in poblacion]
print(f"Costos de la población: {costos}")
print(f"Mejor costo disponible: {min(costos)}")

# Seleccionar un ganador por torneo
ganador = seleccion_torneo(poblacion)
print(f"Costo del ganador:      {calcular_costo(ganador)}")
```

```
Costos de la población: [44, 44, 44, 47, 45, 46, 44, 43, 48, 48]
Mejor costo disponible: 43
Costo del ganador:      43
```

En esta fase a pesar que el ganador tiene costo de 43, (no 41 que es el minímo global), debido a que el torneo selecciona K = 3 individuos al azar y elige el mejor entre ellos. Manteniendo la diversidad de la población y evitando que el GA converja hacia una única solución.

Cruce

El cruce combina dos soluciones padre para generar un hijo que hereda características de ambos.

```
Se utiliza cruce de un punto:
  1. Se elige un punto de corte aleatorio
  2. El hijo toma la primera parte del Padre 1
  3. Y la segunda parte del Padre 2
  4. Si el hijo viola la restricción (se repara)
```

```
def cruce(padre1, padre2):
    """
    Cruce de un punto: combina dos soluciones y repara si viola restricciones.
    """
    punto = random.randint(1, N_TOMAS - 1)
    hijo = padre1[:punto] + padre2[punto:]
    hijo = reparar(hijo)
    return hijo

def reparar(solucion):
    """
    Si algún día supera MAX_TOMAS_DIA tomas, mueve el exceso a otro día.
    """
    solucion = list(solucion)
    conteo = {}
    for dia in solucion:
        conteo[dia] = conteo.get(dia, 0) + 1

    dia_nuevo = max(conteo.keys()) + 1

    for i in range(N_TOMAS):
        dia = solucion[i]
        if conteo[dia] > MAX_TOMAS_DIA:
            # Buscar día con espacio
            movido = False
            for d in sorted(conteo.keys()):
                if d != dia and conteo[d] < MAX_TOMAS_DIA:
                    conteo[dia] -= 1
                    solucion[i] = d
                    conteo[d] += 1
                    movido = True
                    break
            if not movido:
                # Crear día nuevo
                conteo[dia] -= 1
                solucion[i] = dia_nuevo
                conteo[dia_nuevo] = 1
                dia_nuevo += 1

    return solucion
```

```
# Test
padre1 = generar_solucion()
padre2 = generar_solucion()
hijo   = cruce(padre1, padre2)

print(f"Padre 1: {padre1}")
print(f"Padre 2: {padre2}")
print(f"Hijo:    {hijo}")
print()
print(f"Costo Padre 1: {calcular_costo(padre1)}")
print(f"Costo Padre 2: {calcular_costo(padre2)}")
print(f"Costo Hijo:    {calcular_costo(hijo)}")
print(f"¿Hijo válido?: {es_valida(hijo)}")

```

```
Padre 1: [3, 2, 1, 1, 5, 3, 4, 5, 3, 1, 2, 3, 1, 2, 5, 3, 4, 1, 3, 2, 5, 5, 1, 2, 2, 4, 4, 5, 4, 4]
Padre 2: [5, 5, 4, 2, 1, 2, 3, 1, 3, 5, 1, 4, 4, 3, 2, 1, 2, 1, 5, 5, 2, 1, 3, 4, 3, 4, 2, 3, 5, 4]
```

```
Hijo:    [4, 2, 4, 1, 5, 5, 4, 5, 3, 1, 2, 3, 1, 3, 2, 1, 2, 1, 5, 5, 2, 1, 3, 4, 3, 4, 2, 3, 5, 4]

Costo Padre 1: 43
Costo Padre 2: 46
Costo Hijo:    45
¿Hijo válido?: True
```

El hijo generado tiene costo 45, valor intermedio entre el Padre 1 (43) y el Padre 2 (46). El cruce combina partes de ambos padres, heredando características de cada uno. La solución resultante es válida, lo que confirma que la función reparar() funciona correctamente. A lo largo de las generaciones, el GA irá encontrando combinaciones cada vez mejores.

Mutación

La mutación introduce variabilidad aleatoria en las soluciones, evitando que toda la población converja hacia el mismo punto.

```
Con una probabilidad se modifica una solución:
1. Se elige una toma aleatoria
2. Se mueve a un día diferente con espacio disponible
```

```python
def mutar(solucion, prob_mutacion=0.1):
    solucion = list(solucion)

    if random.random() < prob_mutacion:
        toma   = random.randint(0, N_TOMAS - 1)

        conteo = {}
        for dia in solucion:
            conteo[dia] = conteo.get(dia, 0) + 1

        dias_disponibles = [d for d, c in conteo.items()
                            if c < MAX_TOMAS_DIA]

        # Si no hay días disponibles → crear uno nuevo
        if not dias_disponibles:
            dia_nuevo = max(conteo.keys()) + 1
            solucion[toma] = dia_nuevo
        else:
            solucion[toma] = random.choice(dias_disponibles)

    return solucion
```

```python
# Test

sol_original = generar_solucion()
sol_mutada   = mutar(sol_original, prob_mutacion=1.0)

print(f"Original: {sol_original}")
print(f"Mutada:   {sol_mutada}")
print()

# Encontrar qué posición cambió
cambios = [i for i in range(N_TOMAS)
           if sol_original[i] != sol_mutada[i]]
print(f"Posición que cambió (toma): {cambios}")
for i in cambios:
    print(f"  Toma {i+1}: Día {sol_original[i]} → Día {sol_mutada[i]}")

print()
print(f"Costo original: {calcular_costo(sol_original)}")
print(f"Costo mutada:   {calcular_costo(sol_mutada)}")
print(f"¿Mutada válida?: {es_valida(sol_mutada)}")
```

```
Original: [5, 2, 1, 3, 4, 5, 4, 1, 3, 3, 3, 5, 4, 5, 4, 2, 4, 1, 2, 1, 1, 2, 4, 2, 5, 5, 3, 3, 1, 2]
Mutada:   [5, 2, 1, 3, 4, 5, 4, 1, 3, 3, 3, 5, 6, 5, 4, 2, 4, 1, 2, 1, 1, 2, 4, 2, 5, 5, 3, 3, 1, 2]

Posición que cambió (toma): [12]
  Toma 13: Día 4 → Día 6

Costo original: 48
Costo mutada:   51
¿Mutada válida?: True
```

La mutación movió la Toma 29 del Día 1 al Día 6 (nuevo). El costo empeoró de 46 a 48 porque la toma quedó sola en un día nuevo, sin compartir actores con otras tomas.

La mutación no busca mejorar directamente, sino introducir variabilidad para que el GA explore nuevas regiones del espacio de búsqueda. El GA buscará seleccionar las mejores soluciones en cada generación.

Búsqueda local

La búsqueda local intenta mejorar una solución intercambiando los días de dos tomas aleatorias:

```
1. Se eligen dos tomas al azar
2. Se intercambian sus días asignados
3. Si el nuevo costo es menor se acepta
```

```
4. Si no mejora se descarta
5. Se repite  n intentos veces
```

```python
def busqueda_local(solucion, intentos=20):
    """
    Intenta mejorar la solución intercambiando tomas entre días.
    Inspirado en el pairwise interchange de Nordström & Tufekci (1994).
    """
    mejor = list(solucion)
    mejor_costo = calcular_costo(mejor)

    for _ in range(intentos):
        # Elegir dos tomas al azar e intercambiar sus días
        i, j = random.sample(range(N_TOMAS), 2)
        vecino = list(mejor)
        vecino[i], vecino[j] = vecino[j], vecino[i]

        costo_vecino = calcular_costo(vecino)
        if costo_vecino < mejor_costo:
            mejor = vecino
            mejor_costo = costo_vecino

    return mejor

```

```python
#Tests
sol          = generar_solucion()
sol_mejorada = busqueda_local(sol, intentos=20)

print(f"Antes  → Costo: {calcular_costo(sol)}")
print(f"Después → Costo: {calcular_costo(sol_mejorada)}")
print(f"Mejora: {calcular_costo(sol) - calcular_costo(sol_mejorada)} unidades")
print(f"¿Válida?: {es_valida(sol_mejorada)}")

sol_mejorada_100 = busqueda_local(sol, intentos=100)
print(f"\nCon 100 intentos → Costo: {calcular_costo(sol_mejorada_100)}")
print(f"Mejora: {calcular_costo(sol) - calcular_costo(sol_mejorada_100)} unidades")
```

```
Antes  → Costo: 43
Después → Costo: 39
Mejora: 4 unidades
¿Válida?: True

Con 100 intentos → Costo: 36
Mejora: 7 unidades
```

Podemos ver que con 20 intentos de intercambio, la búsqueda local logró reducir el costo de 46 a 38, una mejora de 8 unidades sin necesidad de generar una nueva solución desde cero, cuando se hace con 100 intentos se mantuvo en 38, indicando que con 20 intentos ya se encontró un minímo local.

Pipeline de Algoritmo Genético completo

Una vez implemenatados todos los operadores, se integran en un ciclo evolutivo que se repite durante 500 generaciones, hasta encontrar la planificación de menor costo.

```python
def algoritmo_genetico(
    tam_poblacion=100,
    generaciones=500,
    prob_cruce=0.8,
    prob_mutacion=0.1,
    elitismo=2,
    usar_busqueda_local=True
):
    # --- Inicialización ---
    poblacion = generar_poblacion(tam_poblacion)
    historial_mejor = []
    historial_promedio = []

    mejor_global = min(poblacion, key=calcular_costo)
    mejor_costo_global = calcular_costo(mejor_global)

    print(f"Costo inicial (mejor de población): {mejor_costo_global}")

    for gen in range(generaciones):
        nueva_poblacion = []

        # --- Elitismo: conservar los mejores ---
        poblacion_ordenada = sorted(poblacion, key=calcular_costo)
        nueva_poblacion.extend(poblacion_ordenada[:elitismo])

        # --- Generar el resto de la nueva población ---
        while len(nueva_poblacion) < tam_poblacion:
            # Selección
            padre1 = seleccion_torneo(poblacion)
            padre2 = seleccion_torneo(poblacion)

            # Cruce
            if random.random() < prob_cruce:
                hijo = cruce(padre1, padre2)
            else:
```

```
                hijo = list(padre1)

            # Mutación
            hijo = mutar(hijo, prob_mutacion)

            # Búsqueda local (mejora memética)
            if usar_busqueda_local:
                hijo = busqueda_local(hijo)

            nueva_poblacion.append(hijo)

        poblacion = nueva_poblacion

        # --- Registrar estadísticas ---
        costos = [calcular_costo(ind) for ind in poblacion]
        mejor_gen = min(costos)
        prom_gen  = sum(costos) / len(costos)
        historial_mejor.append(mejor_gen)
        historial_promedio.append(prom_gen)

        if mejor_gen < mejor_costo_global:
            mejor_costo_global = mejor_gen
            mejor_global = poblacion[costos.index(mejor_gen)]

        # Imprimir progreso cada 100 generaciones
        if (gen + 1) % 100 == 0:
            print(f"Gen {gen+1:4d} | Mejor: {mejor_gen} | Promedio: {prom_gen:.1f}")

    print(f"\n Costo final óptimo encontrado: {mejor_costo_global}")
    return mejor_global, mejor_costo_global, historial_mejor, historial_promedio
```

```
#Test

random.seed(42)
np.random.seed(42)

mejor_sol, mejor_costo, hist_mejor, hist_prom = algoritmo_genetico(
    tam_poblacion=100,
    generaciones=500,
    prob_cruce=0.8,
    prob_mutacion=0.1,
    elitismo=2
)
```

```
Costo inicial (mejor de población): 41
Gen  100 | Mejor: 31 | Promedio: 31.4
Gen  200 | Mejor: 31 | Promedio: 31.4
Gen  300 | Mejor: 31 | Promedio: 31.1
Gen  400 | Mejor: 31 | Promedio: 31.5
Gen  500 | Mejor: 31 | Promedio: 31.4

 Costo final óptimo encontrado: 31
```

El GA encontró su mejor solución muy rápido (antes de la generación 100) y se estabilizó, las 400 generaciones restantes no mejoras, debido a que el algoritmo ya convergió con las 100 primeras.

Visualización de resultados

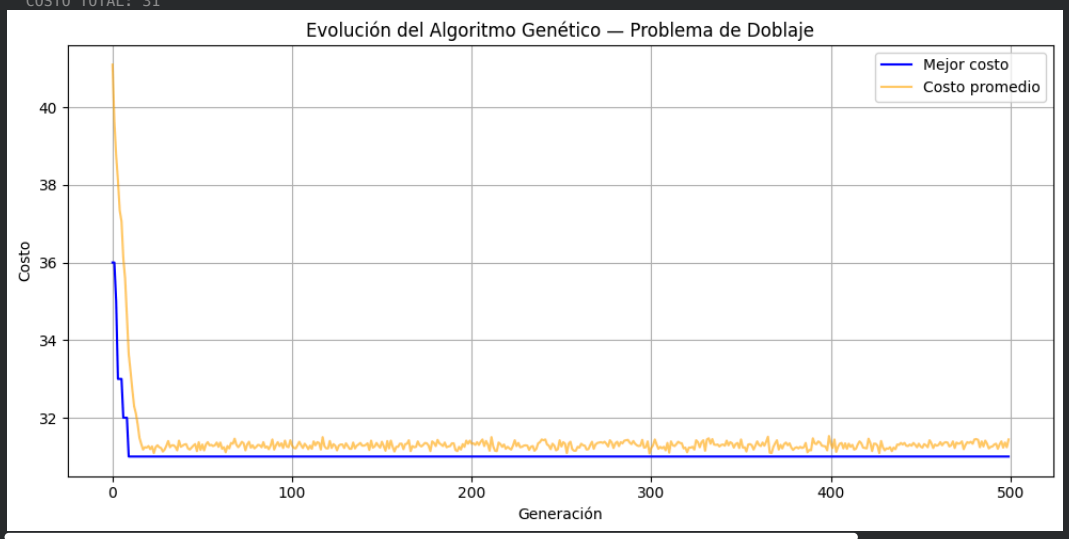
Se muestra la planificación final por día y la evolución del costo a lo larlo de las generaciones

```
def mostrar_solucion(solucion):
    """
    Imprime la asignación de tomas por día y el costo de cada día.
    """
    dias = {}
    for toma, dia in enumerate(solucion):
        if dia not in dias:
            dias[dia] = []
        dias[dia].append(toma + 1)

    print("\nPLANIFICACIÓN POR DÍA:")
    print("-" * 50)
    costo_total = 0
    for dia in sorted(dias.keys()):
        tomas_dia = dias[dia]
        actores_dia = set()
        for t in tomas_dia:
            for actor in range(N_ACTORES):
                if M[t-1][actor] == 1:
                    actores_dia.add(actor + 1)
        costo_dia = len(actores_dia)
        costo_total += costo_dia
        print(f"  Día {dia:2d}: Tomas {tomas_dia} → Actores {sorted(actores_dia)} → Costo: {costo_dia}")
    print("-" * 50)
    print(f"  COSTO TOTAL: {costo_total}")

def graficar_evolucion(historial_mejor, historial_promedio):
    plt.figure(figsize=(10, 5))
    plt.plot(historial_mejor,   label='Mejor costo', color='blue')
    plt.plot(historial_promedio, label='Costo promedio', color='orange', alpha=0.6)
    plt.xlabel('Generación')
    plt.ylabel('Costo')
    plt.title('Evolución del Algoritmo Genético — Problema de Doblaje')
```

```
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

```

```
random.seed(42)
np.random.seed(42)

# Corre el GA
mejor_sol, mejor_costo, hist_mejor, hist_prom = algoritmo_genetico(
    tam_poblacion=100,
    generaciones=500,
    prob_cruce=0.8,
    prob_mutacion=0.1,
    elitismo=2
)

# Muestra planificación por día
mostrar_solucion(mejor_sol)

# Muestra gráfica de evolución
graficar_evolucion(hist_mejor, hist_prom)
```

```
Costo inicial (mejor de población): 41
Gen  100 | Mejor: 31 | Promedio: 31.4
Gen  200 | Mejor: 31 | Promedio: 31.4
Gen  300 | Mejor: 31 | Promedio: 31.1
Gen  400 | Mejor: 31 | Promedio: 31.5
Gen  500 | Mejor: 31 | Promedio: 31.4

 Costo final óptimo encontrado: 31

PLANIFICACIÓN POR DÍA:
--------------------------------------------------
  Día  1: Tomas [2, 4, 5, 6, 7, 14] → Actores [1, 2, 4, 5, 6, 8] → Costo: 6
  Día  2: Tomas [3, 18, 21, 26, 27, 28] → Actores [2, 3, 4, 6, 8, 9, 10] → Costo: 7
  Día  3: Tomas [15, 19, 23, 24, 29, 30] → Actores [1, 3, 5, 7, 9, 10] → Costo: 6
  Día  4: Tomas [1, 8, 11, 16, 17, 25] → Actores [1, 2, 3, 4, 5, 10] → Costo: 6
  Día  5: Tomas [9, 10, 12, 13, 20, 22] → Actores [1, 4, 5, 7, 8, 9] → Costo: 6
--------------------------------------------------
  COSTO TOTAL: 31
```



El GA encontró una planificación de 5 días con un costo total de 31 actor-días, partiendo de soluciones aleatorias con costo promedio de 45. Esto representa una mejora del 24% respecto al punto de partida.

Todas las restricciones se cumplen: cada día tiene exactamente 6 tomas y se utilizan el mínimo de días posible (5).

El Día 2 es el más costoso con 7 actores convocados, mientras los demás días requieren solo 6.

Conclusiones:

- El GA encontró su mejor solución antes de la generación 100, lo que representa solo el 20% del total de generaciones ejecutadas. Las 400 generaciones restantes confirmaron que la solución era estable.
- El GA redujo el costo de 41 (solución aleatoria inicial) a 31 (solución optimizada).
- La solución encontrada utiliza 5 día de grabación, que es el mínimo teórico posible (30 tomas / 6 tomas por día = 5 días), cumpliendo todas las restricciones del problema.
- El GA no garantiza el óptimo global. El costo de 31 es el mejor encontrado en esta ejecución, pero podrían existir soluciones con costo.

Uso de herramientas de IA:

- Se emplearo herramientas de IA como herramienta para la estructuración del enunciado y generación de tablas de datos de entrada en formato Markdown para mejorar la presentación.
- Resolución de dudas a lo largo del pipeline.
- Apoyo en la identificación de errores y ajustes en la implementación.

Referencias:

- A Genetic Algorithm for the Talent Scheduling Problem" Nordström & Tufekci — Computers & Operations Research, 1994